

AlgoC - Documentazione tecnica

Progetto EILP (a.a. 2025/2026)

Sabato Iaquino

0512123029

1. Introduzione al progetto	3
2. Pipeline di compilazione	4
3. Struttura del compilatore	5
4. Dipendenze	6
5. Specifiche lessicale	6
6. Struttura dei programmi	7
7. Tipi e dichiarazioni	7
8. Funzioni e procedure	8
9. I/O	9
10. Strutture di controllo	10
11. Espressioni e operatori	11
12. Statement	12
13. Keyword	13
14. Uso dell'AI nel progetto	13

1. Introduzione al progetto

AlgoC è un **linguaggio di programmazione** progettato per scrivere codice con una **sintassi vicina allo pseudocodice** utilizzato nella specifica degli algoritmi: l'obiettivo è **ridurre la distanza** tra la **rappresentazione teorica** di un algoritmo e la sua **esecuzione pratica**. Un programma scritto in AlgoC può essere tradotto automaticamente in codice C e compilato in un eseguibile.

Il progetto implementa un compilatore completo, formato da:

- * **Grammatica Lark** per generare automaticamente il parser, contenuta in *grammar.lark*.

- * **Analisi lessicale e sintattica** tramite lexer e parser Lark, contenuta in *ast_generator.py*.

- * **Costruzione di AST**, contenuta in *ast_generator.py*, tramite i nodi definiti come classi Python, contenuti in *ast_nodes.py*.

- * **Analisi semantica su AST**, contenuta in *code_generator.py*, tramite l'uso di symbol table a livello singolo, contenuta in *symbol_table.py*.

- * **Generazione del codice C dall'AST**, contenuta in *code_generator.py*.

- * **Driver generale** del compilatore per compilare codice automaticamente da AlgoC a C ad eseguibile tramite gcc¹, oppure svolgere la sola compilazione da AlgoC a C.

- * Le caratteristiche principali del compilatore sono:

- * **Compilazione automatica:** dal codice AlgoC si genera codice C ottimizzato e compilabile.

- * **Supporto all'uso di array:** sono supportati gli array di *int*, *real*, *char* e *boolean*.

- * **Strutture di controllo semplificate:** le strutture di controllo vengono ottimizzate per l'iterazione sugli elementi di una struttura dati.

¹ GNU Compiler Collection, collezione di compilatori ottimizzanti multiplatforma per svariati linguaggi, tra cui il C.

✱ **Assegnazione semplificata:** l'assegnazione viene svolta tramite l'operatore <=.

✱ **Operatori semplificati:** viene rimosso il supporto agli operatori bitwise, di poca utilità in questo campo, e viene semplificato l'uso degli operatori, divisi in:

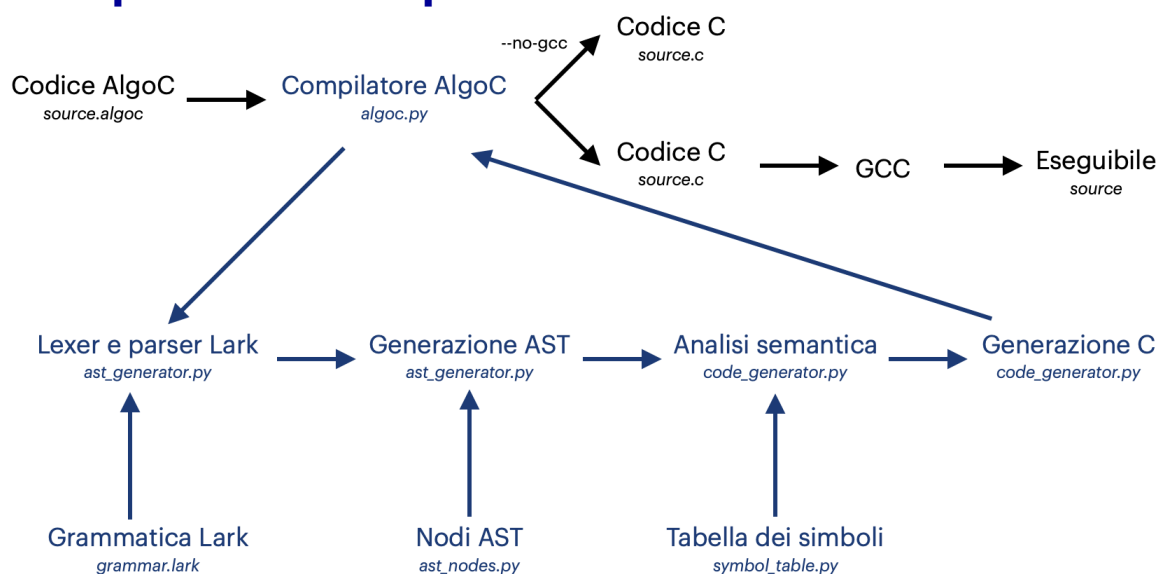
✱ **Aritmetici:** +, -, *, / e %.

✱ **Relazionali:** <, >, <=, >= e =.

✱ **Logici:** |, & e !.

✱ **Passaggi commentabili:** è possibile aggiungere commenti in ogni punto del codice racchiudendo il testo tra %%.

2. Pipeline di compilazione



Pipeline di compilazione del compilatore di AlgoC

Per compilare un sorgente in AlgoC, in un ambiente Python con Lark e GCC, è possibile richiamare da riga di comando **algoc.py source.algoc** per ottenere il sorgente in C e l'eseguibile nella stessa cartella in cui si trova il sorgente, oppure **algoc.py source.algoc --no-gcc²** per ottenere il sorgente in C.

² Qualsiasi chiamata al compilatore di AlgoC in un ambiente senza GCC verrà tradotta in questa forma.

La versione di C utilizzata per la compilazione è C99, questa scelta deriva dal fatto che C99 già implementa la libreria che definisce i booleani³.

3. Struttura del compilatore

Il compilatore per AlgoC è scritto interamente in Python, esclusa la grammatica Lark scritta in EBNF, ed è formato dai seguenti moduli:

- * **`grammar.lark`**, contiene la **grammatica per Lark** in formato EBNF. Le regole grammaticali sono raggruppate per *program structure*, *variable declaration*, *function and procedure definition*, *body of program*, *function and procedure*, *statements*, *control structures*, *expressions*, *array access*, *function calling* e *lexer rules*.

- * **`ast_nodes.py`**, contiene i **nodi** per la costruzione dell'**AST**, raggruppati come nella grammatica.

- * **`ast_generator.py`**, contiene il **caricamento della grammatica**, la creazione del parser Lark, l'**analisi lessicale e semantica** e la **generazione dell'AST** tramite i nodi dichiarati in `ast_nodes.py`.

- * **`symbol_table.py`**, contiene la dichiarazione di un **simbolo**, la struttura della **tabella dei simboli** e l'eccezione generale di AlgoC, **`AlgoCError`**.

- * **`code_generator.py`**, svolge l'**analisi semantica** del programma, tramite un'istanza della tabella dei simboli dichiarata in `symbol_table.py`, e genera il codice in C a partire dall'AST.

- * **`algoc.py`**, è il **driver del compilatore**, viene eseguito da terminale in un ambiente Python con Lark, riceve il file sorgente e produce il sorgente in C, se chiamato con l'argomento `--no-gcc`, e compila automaticamente il sorgente C in un'eseguibile, se non viene specificato l'argomento `--no-gcc`.

³ stdbool.h, <https://pubs.opengroup.org/onlinepubs/009695399/basedefs/stdbool.h.html>

4. Dipendenze

Il compilatore ha due dipendenze:

- * **Lark**, modulo Python general-purpose per il parsing, **necessario** per il funzionamento del compilatore in quanto serve per il lexer e il parser.

- * **GCC** (non necessario ma fortemente consigliato), collezione di compilatori per i linguaggi di programmazione principali, **non necessario** per il funzionamento del compilatore **ma fortemente consigliato**, in quanto necessario per la compilazione dal C. In sua mancanza ogni compilazione verrà effettuata automaticamente con l'opzione `--no-gcc`.

5. Specifiche lessicale

I token riconosciuto dal compilatore sono:

- * **Identificatori**: iniziano con underscore⁴ o una lettera e contengono underscore, lettere o numeri.

- * **Costanti intere**: formate da solo cifre.

- * **Costanti reali**: formate da due insiemi di cifre separate da un punto.

- * **Costanti carattere**: una qualsiasi lettera o escape sequence⁵.

- * **Costanti stringa**: un qualsiasi insieme di costanti carattere racchiuso tra `""`.

- * **Costanti booleane**: `true` o `false`

- * **Commenti**: qualsiasi elemento racchiuso tra `%%`.

- * **Spazi bianchi**: import di `common.ws`, dunque qualsiasi spazio bianco comune riconosciuto da Lark (spazio, tabulazioni...).

Il **lexer ignorerà** tutto ciò che è uno **spazio bianco** o un **commento**.

⁴ Il carattere `_`

⁵ Le escape sequence di AlgoC sono le stesse del C, in quanto il valore delle variabili stringhe e dei caratteri resta invariato durante la compilazione da AlgoC a C.

6. Struttura dei programmi

Un programma è definito come *dichiarazioni globali* **prog** corpo del programma **endprog**.

Le dichiarazioni globali possono essere dichiarazioni di variabili, di funzioni o di procedure, mentre il corpo del programma deve essere racchiuso tra **prog** ed **endprog**, che viene tradotto in C con *int main(void){... return 0;}*.

```
% dichiarazioni globali opzionali %  
  
prog  
    % corpo principale %  
endprog
```

Struttura generale di un programma

7. Tipi e dichiarazioni

I tipi supportati sono:

- * **int**, tradotto in C con *int*.
- * **real**, tradotto in C con *double*.
- * **char**, tradotto in C con *char*.
- * **string**, tradotto in C con un array di *char* di dimensione *STRING_SIZE*, una costante di valore 256 scelta per semplicità.
- * **boolean**, tradotto in C come *bool*.
- * **array**, di tutti i tipi primitivi.

Le dichiarazioni di variabili vengono effettuate nei blocchi **var ... endvar** e ogni variabile è dichiarata come *id, id, ... : type* ; mentre un array è dichiarato nello stesso modo ma aggiungendo la dimensione dell'array racchiusa tra parentesi quadre dopo il tipo.

Tutto ciò che viene dichiarato nel blocco **var ... endvar** viene tradotto nelle corrispondenti dichiarazioni in C, raggruppando dichiarazioni sotto lo stesso tipo nella stessa dichiarazione.

L'assegnazione non può essere svolta in fase di dichiarazione della variabile, va svolta successivamente ed è possibile effettuare solo assegnazioni singole. Un'assegnazione è svolta come *var <- expression ;* dove <- è l'operatore di assegnazione, viene tradotta in C come *var = expression ;* per *int*, *real*, *char* e *boolean*, mentre per le stringhe l'assegnazione viene fatta tramite la funzione ***static void algoc_assign_string(char *dest, const char *src)*** che richiama la funzione ***strncpy*** sulla stringa da inserire nella variabile e aggiunge \0 alla fine.

```
var
  x: int;
  y: real;
  nome: string;
endvar
```

Esempio di
dichiarazioni singole

```
var
  a, b, c: int;
endvar
```

Esempio di
dichiarazioni multiple
con singolo tipo

```
var
  v: int[10];
endvar
```

Esempio di
dichiarazione di array

8. Funzioni e procedure

È possibile definire ed utilizzare funzioni e procedure, le prime si differenziano dalle seconde perché ritornano un valore, mentre le seconde no.

Una funzione è definita come ***func id (parameters) -> return type : body endfunc***, mentre una procedura è definita come ***proc id (parameters) : body endproc***.

I parametri sono definiti singolarmente come *id : type*, ma possono essere molteplici e vengono separati da una virgola, e il corpo può contenere dichiarazioni di variabili, statement e chiamate ad altre funzioni.

Funzioni e procedure devono essere dichiarate all'esterno del corpo del programma, non possono essere dichiarate funzioni o procedure all'interno di altre funzioni o procedure.

Funzioni e procedure vedono tradotte nelle corrispettive definizioni in C, la struttura uguale alle classiche funzioni in C, la differenza sta nel tipo di

ritorno: per le funzioni viene specificato il tipo di ritorno indicato, per le procedure viene specificato il tipo di ritorno *void*.

Le funzioni devono ritornare un valore con *return expression* ;

Una funzione può essere chiamata in qualunque punto del programma tramite *id (param) ;*.

```
func somma(a: int, b: int) -> int:
    return a + b;
endfunc
```

Esempio di funzione

```
proc printHelloWorld():
    --> "Hello, World";
endproc
```

Esempio di procedura

9. I/O

È possibile svolgere operazioni di I/O formattato tramite l'operatore di input *--> argument , ... ;* e di output *<-- argument , ... ;* . Le operazioni di I/O vengono tradotte in C con le funzioni ***printf*** e ***scanf***.

Per svolgere operazioni di I/O sulle variabili bisogna utilizzare l'operatore di formattazione sulle variabili *\${ argument }*, in modo che quando il codice viene tradotto in C il compilatore sappia la specifica di conversione corretta da utilizzare. Con le stringhe in input viene effettuata la lettura con la specifica *%255s*, in modo da assicurarsi che il 256esimo carattere sia *\0*.

```
--> "What's your name? ";
<-- $(name);
--> "Hi " $(name) ", it's nice to meet you!";
```

Esempio di uso di I/O formattato

10.Strutture di controllo

È possibile utilizzare diverse strutture di controllo per modificare il normale flusso di esecuzione del programma.

Esiste la struttura condizionale **if condition then body elseif condition then body ... else body endif ;** in cui è possibile seguire diversi flussi di esecuzione in base alle condizioni espresse. È possibile concatenare più strutture condizionali aggiungendo blocchi **elseif condition then body**. Una struttura condizionale deve terminare con il blocco **else endif**. Viene tradotta nella struttura **if ... else if ... else** corrispondente del C.

```
if (number > 0) then
    --> "Il valore è positivo" ;
elseif (number < 0) then
    --> "Il valore è negativo" ;
else
    --> "Il valore è 0" ;
endif;
```

Esempio di struttura condizionale

Esiste il ciclo **while condition do body endwhile ;** per ciclare finché l'espressione condizionale indicata è vera⁶. Viene tradotto nella struttura **while** corrispondente del C.

```
while (number < 10) do
    --> $(number) ;
    number <- number + 1 ;
endwhile ;
```

Esempio di ciclo **while**

Esiste il ciclo **do body while condition ;** per ciclare almeno una volta e finché l'espressione condizionale indicata è vera. Viene tradotto nella struttura **do ... while** corrispondente in C.

⁶ Se l'espressione è falsa in partenza, il ciclo non viene mai eseguito.

```
do
  --> $(number) ;
  number <- number + 1 ;
while (number < 10) ;
```

Esempio di ciclo **do while**

Esiste il ciclo **for initial expression to final do body endfor** ; per ciclare un numero di volte pari a *final - initial*. Il ciclo richiede in input un'espressione iniziale, formata da un'operazione di assegnazione di un valore ad una variabile iteratore⁷, e un valore finale, ad ogni iterazione il ciclo incrementa la variabile, quando la variabile iteratore ha superato il valore finale indicato. Viene tradotto nella struttura **for (var = initial ; var <= final ; var ++)** corrispondente in C.

```
var
  n : int ;
endvar

for n <- 1 to 10 do
  --> $(n)
endfor ;
```

Esempio di ciclo **for**

11. Espressioni e operatori

Le espressioni vengono elaborate con la precedenza classica degli operatori in C, pertanto gli operatori vengono elaborati per precedenza decrescente come:

1. Parentesi, chiamate a funzione o procedura, accesso ad elementi degli array.
2. Operatori unari.
3. Moltiplicazioni, divisioni e moduli.
4. Somme e sottrazioni.
5. Confronti.

⁷ La variabile iteratore deve essere dichiarata prima di essere utilizzata nel ciclo.

6. AND.

7. OR.

Gli operatori di AlgoC sono:

✱ Operatore di assegnazione <=.

✱ Operatori aritmetici somma +, sottrazione -, moltiplicazione *, divisione / e modulo %. Somma, sottrazione e moltiplicazione richiedono operandi numerici restituendo lo stesso tipo di operando, la divisione richiede operandi numerici e restituisce *real*, il modulo richiede operandi *int* e restituisce *int*.

✱ Operatori relazionali minore <, minore o uguale <=, maggiore >, maggiore o uguale >=, uguale = e diverso <>. Minore, minore o uguale, maggiore e maggiore o uguale confrontano valori numerici e restituiscono *boolean*, gli operatori uguale e diverso confrontano valori compatibili e restituiscono *boolean*.

✱ Operatori logici AND &, OR | e NOT !, richiedono tutti operandi booleani.

Gli operatori di AlgoC si traducono in C come:

AlgoC	C
<=	=
+	+
-	-
/	/
*	*

AlgoC	C
%	%
<	<
<=	<=
>	>
>=	>=

AlgoC	C
=	==
<>	!=
&	&&
!	!

12.Statement

È importante che alla fine di ogni statement ci sia ; che è fondamentale per la chiusura dello statement a livello sintattico. Gli statement che devono terminare con ; sono:

✱ Dichiarazione di variabili.

- * Assegnazioni.
- * Chiamate a funzioni o procedure.
- * Operazioni di I/O.
- * Strutture di controllo e cicli.

13.Keyword

AlgoC presenta le seguenti keyword, non utilizzabili come identificatori:

prog	endprog	var	endvar	func
endfunc	proc	endproc	return	if
then	elseif	else	endif	while
do	endwhile	for	to	endfor
int	real	char	string	boolean
true	false			

14.Uso dell'AI nel progetto

Durante la realizzazione del progetto è stata utilizzata l'AI come strumento di verifica, suggerimento e generazione, in particolare è stata utilizzata:

- Per chiarimenti riguardante la struttura di un compilatore con Python e Lark.
- Spiegazione teoria della definizione e del funzionamento di una Symbol Table.
- Generazione di alcuni esempi di programmi in AlgoC (calculator.algoc e array.algoc).
- Generazione di alcune porzioni di codice ripetitive e già definite.

L'AI non ha sostituito la scrittura del compilatore ma l'ha accompagnata suggerendo moduli e strutture del codice, rappresentando un grande vantaggio nella velocità di produzione delle bozze e nella chiarezza della consegna.

Il principale svantaggio individuato nell'uso dell'AI è stato far capire la grandezza del progetto, molto contenuta, rispetto alle idee di compilatore che l'AI credeva si dovesse sviluppare, pertanto suggeriva spesso di utilizzare linguaggio di basso livello o di implementare funzionalità del compilatore di gran lunga superflue.